SYSTEM ANALYSIS DOCUMENT

DaddiesTrip — AI-Powered Cross-Border Travel Orchestration Platform

SugarDaddies: We take care of everything. Especially you.

Document Title	System Analysis Document
Project Name	DaddiesTrip — AI-Powered Travel Orchestration Platform
Team / Group Name	SugarDaddies
Group Motto	SugarDaddies: We take care of everything. Especially you.
Product Tagline	Sit back, relax, let daddy handle the itinerary.
Hackathon	UM Hackathon 2026
Institution	Universiti Teknologi Malaysia (UTM)
Live Demo	https://daddies-trip.vercel.app/
Repository	github.com/SkyLee310/UMHackathon-DaddiesTrip
Document Date	April 25, 2026

1. Executive Summary

DaddiesTrip is a full-stack, AI-powered group travel planning platform developed by team SugarDaddies for UM Hackathon 2026. The system eliminates the fragmentation of traditional group travel planning by providing a single conversational interface that orchestrates the complete lifecycle of a trip — from natural language prompt to actionable itinerary — within seconds.

The platform addresses a well-known pain point: travellers currently must juggle 4–5 separate applications to draft itineraries, compare flights, find accommodations, and split costs across currencies. DaddiesTrip collapses this entire workflow into one AI-driven pipeline.

> *Tagline: "Sit back, relax, let daddy handle the itinerary." — DaddiesTrip automates itinerary generation, flight and hotel discovery, multi-currency expense splitting, and simulated payment settlement through a real-time streaming interface.*

2. Problem Statement

2.1 Current Landscape

Group travel planning is inherently fragmented. A typical group of friends or colleagues must navigate:

- Multiple itinerary drafting tools (Google Docs, Notion, spreadsheets)
- Flight comparison engines (Google Flights, Skyscanner, AirAsia)
- Accommodation platforms (Booking.com, Agoda, Airbnb)
- Manual multi-currency expense calculations across bank rates
- Messaging apps to coordinate preferences among group members

2.2 Key Pain Points

Pain Point	Impact
Fragmented tooling	Context-switching across 4–5 apps increases cognitive load and planning time
Manual itinerary drafting	Non-experts struggle to create optimal day-by-day schedules
Flight & hotel research	Price comparison requires visiting multiple platforms independently
Multi-currency splitting	Manual FX calculations introduce errors and disputes
Group coordination	Preference aggregation across members has no unified interface

3. System Objectives

- Provide a single conversational interface for end-to-end group travel planning
- Automate itinerary generation using a multi-agent LLM pipeline
- Surface real flight options and hotel recommendations with pricing
- Deliver real-time progressive feedback via Server-Sent Events (SSE)
- Perform deterministic multi-currency expense splitting with live exchange rates
- Support itinerary customisation through natural language amendment
- Ensure robustness via automated hallucination detection and correction
- Export the final itinerary as a downloadable PDF

4. System Scope

4.1 In Scope

- Conversational trip planning via text and voice (Web Speech API)
- AI-generated day-by-day itineraries with activities, food, and transport
- Flight and hotel enrichment with realistic pricing and links
- Multi-currency expense splitting with live FX rates (17+ currencies)
- Interactive Google Maps embedding for each activity location
- Natural language amendment of individual itinerary items
- Simulated card payment and booking confirmation flow
- PDF export of the complete itinerary
- Responsive web UI (mobile, tablet, desktop)

4.2 Out of Scope

- Real payment processing (Stripe, FPX, etc.) — simulated only
- Live flight inventory API (IATA/GDS) — LLM-enriched with Skyscanner/Google Flights deep-links
- User authentication and persistent trip history
- Native mobile applications (iOS / Android)
- Real-time group collaboration and chat

5. System Architecture

5.1 Architectural Overview

DaddiesTrip adopts a split-deployment architecture: the React SPA is hosted on Vercel's CDN edge network while the FastAPI backend runs as a persistent Docker container on Railway. This separation was a deliberate design decision driven by the pipeline's runtime characteristics — LLM agent pipelines routinely take 30–90 seconds, which exceeds Vercel's serverless function timeout limits.

> *Architecture Pattern: Client → CDN (Vercel) → Persistent Container (Railway) → Multi-Agent LLM Pipeline → SSE Stream → Client*

5.2 Deployment Topology

Layer	Technology & Rationale
Frontend CDN	Vercel — global edge delivery, automatic HTTPS, instant cache invalidation
Backend Host	Railway — persistent Docker containers with no hard timeout limits
Container Runtime	Docker (multi-stage build) — reproducible environment, dependency isolation
API Protocol	FastAPI + SSE — async Python, automatic OpenAPI docs, streaming support
LLM Provider	Ilmu AI API (OpenAI-compatible) — Malaysian LLM service (ilmu-glm-5.1 model)
Frontend Framework	React 18 + Vite 5 — fast HMR, tree-shaken production build
Styling	Tailwind CSS 3 — utility-first, responsive without custom CSS overhead
Currency API	@fawazahmed0/currency-api — free, offline-capable FX rates for 17+ currencies
Maps	Google Maps Embed API — zero-install, iframe-based location visualisation

5.3 Request Lifecycle

The following describes the sequence of events from the moment a user submits a travel prompt:

- User enters a natural language prompt (e.g., "5 days in Tokyo for 4 people under RM5000")
- The frontend opens an SSE connection to POST /api/plan-trip-stream
- The Orchestrator on Railway invokes the four-agent pipeline sequentially
- Each completed agent stage emits an SSE event to the client in real time

- The frontend progressively renders itinerary cards, flight options, and cost summaries
- The Edge Agent validates the final payload and emits the complete event
- The user can then amend items, view maps, split costs, or download a PDF

6. Multi-Agent AI Pipeline

6.1 Design Philosophy

The pipeline is designed around strict task decoupling: each agent is responsible for exactly one concern. This minimises LLM hallucinations (no single agent must know too much), enables parallel optimisation of prompts per task, and produces deterministic SSE checkpoints the frontend can render progressively.

6.2 Agent Definitions

Analyzer Agent *[LLM + Regex]*
Validates that the user prompt contains all four required fields: destination, dates, number of participants, and budget. If any field is absent, it returns a clarification event with a list of missing fields and halts the pipeline. On success, it extracts structured metadata and passes it downstream.

Planner Agent *[LLM (GPT-style)]*
Consumes the extracted metadata and generates a complete day-by-day itinerary skeleton. Output includes activities per day, food recommendations, transport vectors between locations, weather advice, and packing tips. Emits a partial_itinerary SSE event so the user sees results immediately.

Booking Agent *[LLM (Enrichment)]*
Takes the draft itinerary and enriches it with real-world metadata: hotel names, star ratings, approximate MYR costs, airline options with departure/arrival times, and Skyscanner/Google Flights deep-links. Emits a partial_flights SSE event upon completion. Also handles per-item amendment requests via /api/amend-item.

Edge Agent *[Python Heuristics (Deterministic)]*
Acts as a QA layer that detects and corrects common LLM failure modes without invoking another LLM call. Specific checks include: RM25 cost repetition bug (all costs identical), same-airport round-trip flights, missing day numbers or location fields, and invalid flight routes. Corrections are applied in pure Python and the final validated payload is emitted as the complete SSE event.

6.3 Orchestrator

The OrchestratorAgent (mock_agents.py) coordinates the four agents sequentially and handles cross-cutting concerns:

- Input truncation: prompts exceeding 1,500 words are trimmed before reaching the LLM
- Budget aggregation: pure Python computation (no LLM) summing all costs
- Currency conversion: live FX fetch with deterministic offline fallback
- SSE emission: each agent completion triggers a typed event to the frontend

6.4 Pipeline SSE Event Schema

Event Type	Trigger	Payload Fields
progress	New pipeline stage begins	{ text }
clarification	Analyzer detects missing fields	{ message, missing_fields }
partial_itinerary	Planner Agent completes	{ days, num_participants }
partial_flights	Booking Agent resolves transport	{ flight_options, num_participants }
complete	Edge Agent validates final output	{ data: FullTripObject }
error	Fatal pipeline failure	{ message }

7. Functional Requirements

FR-ID	Feature	Priority	Description
FR-01	Conversational Trip Planning	Must Have	Accept free-text prompts describing destination, dates, group size, and budget
FR-02	Voice Input	Should Have	Web Speech API integration for hands-free prompt entry with real-time transcription
FR-03	Prompt Validation	Must Have	Analyzer Agent enforces presence of all 4 required fields before proceeding
FR-04	Itinerary Generation	Must Have	Day-by-day schedule with activities, food, transport, and weather advice
FR-05	Flight Options	Must Have	Real airline options with pricing, times, and booking deep-links
FR-06	Hotel Recommendations	Must Have	Named hotels with star ratings, costs, and availability context
FR-07	Real-Time SSE Streaming	Must Have	Progressive frontend rendering as each agent stage completes
FR-08	Item Amendment	Must Have	Natural language customisation of individual hotels, restaurants, or activities
FR-09	Google Maps Embedding	Should Have	Inline Google Maps iframe per activity for immediate geographic context
FR-10	Multi-Currency Splitting	Must Have	Automatic cost division with live FX rates for 17+ currencies
FR-11	Payment Simulation	Should Have	Card payment flow with visual card preview and booking confirmation
FR-12	PDF Export	Should Have	One-click itinerary download as a printable PDF
FR-13	Hallucination Correction	Must Have	Edge Agent detects and corrects common LLM output errors deterministically
FR-14	Offline FX Fallback	Must Have	Static exchange rate table used when live currency API is unavailable

8. Non-Functional Requirements

NFR-ID	Category	Requirement
NFR-01	Performance	First SSE partial_itinerary event must arrive within 15 seconds of prompt submission
NFR-02	Performance	Complete pipeline (all 4 agents) must finish within 90 seconds for standard prompts

NFR-ID	Category	Requirement
NFR-03	Availability	Railway backend must maintain 99%+ uptime during hackathon evaluation window
NFR-04	Scalability	Frontend CDN (Vercel) must handle concurrent user sessions without degradation
NFR-05	Reliability	Edge Agent must catch and correct the RM25 repetition bug and same-airport route errors 100% of the time
NFR-06	Security	API keys must never be committed to the repository or exposed to the client
NFR-07	Security	Card input fields must use client-side validation only (no real card data stored)
NFR-08	Usability	UI must be fully functional on mobile (iOS Safari, Android Chrome) without horizontal scrolling
NFR-09	Maintainability	Each agent must be a self-contained module with a single-responsibility interface
NFR-10	Portability	Docker build must produce an identical environment across Railway and local development

9. API Reference

9.1 Endpoints

Method	Endpoint	Auth	Description
POST	/api/plan-trip-stream	API Key (server-side)	Primary pipeline trigger — streams results via SSE
POST	/api/amend-item	API Key (server-side)	Amend a specific hotel, food, or activity item using natural language
POST	/api/settle	None	Simulate group ledger card payment settlement
GET	/api/health	None	Service health check — returns 200 OK when backend is ready

9.2 Request / Response Schemas

/api/plan-trip-stream — Request

{ "prompt": "5 days in Tokyo for 4 people under RM5000" }

/api/amend-item — Request

{ "item_type": "hotel", "current_item": { "name": "Hotel A", "cost_myr": 300 }, "user_preference": "cheaper, near train station", "trip_summary": { "destination": "Tokyo", "budget_myr": 5000 } }

/api/settle — Request

{ "group_id": "group_123", "user_id": "user_1", "card_number": "4111111111111111" }

10. Data Flow & Integration

10.1 External Integrations

External Service	Integration Method & Purpose
Ilmu AI API	REST (OpenAI-compatible) — LLM inference for all four agents
@fawazahmed0/currency-api	CDN-hosted JSON — live FX rates for 170+ currencies, offline fallback for 17

External Service	Integration Method & Purpose
Google Maps Embed API	iframe embed — activity location visualisation, no server-side key required
Skyscanner (deep-link)	URL template — pre-filled flight search links from booking agent output
Google Flights (deep-link)	URL template — alternative flight search links from booking agent output

10.2 Internal Data Structures

- FullTripObject — root output of the pipeline: { days[], flights[], hotel, budget, currency_rates, split_per_person }
- Day — { day_number, title, activities[], food[], transport, weather_note }
- Activity — { name, location, cost_myr, duration_min, maps_query }
- Flight — { airline, flight_number, departure, arrival, cost_myr, skyscanner_url }
- Hotel — { name, stars, cost_per_night_myr, check_in, check_out, total_cost_myr }
- BudgetSummary — { total_cost_myr, budget_myr, surplus_or_deficit, saving_tips[] }

11. Technology Stack

Layer	Technology	Version	Role
Frontend	React	18.x	Component-based SPA rendering
Frontend	Vite	5.x	Build tool with fast HMR
Frontend	Tailwind CSS	3.x	Utility-first responsive styling
Frontend	Lucide React	Latest	Icon library
Backend	FastAPI	Latest	Async Python web framework
Backend	Python	3.10+	Agent logic, data processing
Backend	Pydantic	v2	Schema validation
Backend	Pandas	Latest	Expense aggregation
AI/LLM	Ilmu AI API	ilmu-glm-5.1	LLM inference (OpenAI-compat.)
Infra	Railway	—	Backend persistent container host
Infra	Vercel	—	Frontend CDN
Infra	Docker	Multi-stage	Build reproducibility
Currency	fawazahmed0	CDN	Live FX rates + offline fallback
Maps	Google Maps	Embed	Activity location iframes
Voice	Web Speech API	Browser	Hands-free voice input
Language	JavaScript	ES2022	55% of codebase (frontend)
Language	Python	3.10+	24% of codebase (backend)
Language	CSS	3	15% of codebase (styles)

12. Testing Strategy

12.1 Test Suite

The project includes a PyTest suite in backend/tests/test_agents.py covering end-to-end pipeline validation, negative payment cases, and input safety. Tests are run with:

```
python -m pytest backend/tests/test_agents.py -v
```

Test ID	Category	Description
TC-01	Integration	End-to-end SSE pipeline — validates complete schema (itinerary, flights, budget, split) from a valid prompt

Test ID	Category	Description
TC-02	Negative / Security	Deterministic ledger rejection — verifies invalid card numbers are rejected by the settle endpoint
AI-01	Safety / Robustness	Buffer overflow protection — ensures oversized prompts are truncated to 1,500 words before reaching the LLM

12.2 Hallucination Test Coverage

The Edge Agent's heuristic checks constitute an implicit regression suite for known LLM failure modes:

- RM25 cost repetition bug — all costs in the itinerary are identical at RM25
- Same-airport round-trip routing — departure and return airports are the same
- Missing required fields — day numbers, location names, or cost fields are null
- Invalid flight routes — origin city matches destination city

13. Project Structure

Path	Description
backend/agents/base_agent.py	LLM client abstraction, JSON repair utility, retry logic with exponential back-off
backend/agents/analyzer_agent.py	Prompt validation and field extraction (destination, dates, participants, budget)
backend/agents/planner_agent.py	Day-by-day itinerary generation with activities, food, and transport vectors
backend/agents/booking_agent.py	Flight/hotel/activity enrichment and per-item amendment handler
backend/agents/edge_agent.py	Deterministic hallucination detection and correction layer
backend/agents/mock_agents.py	OrchestratorAgent — sequential pipeline coordinator and SSE emitter
backend/ledger/ledger_service.py	Currency conversion (live + offline), budget aggregation, payment simulation
backend/main.py	FastAPI application — defines all four API endpoints and SSE stream logic
backend/tests/test_agents.py	PyTest suite — TC-01, TC-02, AI-01
frontend/App.jsx	Main React application — all UI state, SSE client, component rendering
frontend/main.jsx	React entry point

Path	Description
api/index.py	Vercel Serverless Function entry point (used for health checks / edge routing)
Dockerfile	Multi-stage Docker build for Railway deployment
vercel.json	Vercel routing config — proxies /api/* to Railway backend URL
.env.example	Environment variable template (Z_AI_API_KEY, Z_AI_BASE_URL, Z_AI_MODEL, VITE_API_BASE_URL)

14. Known Limitations & Risks

Limitation / Risk	Mitigation
LLM pipeline latency (30–90s)	SSE streaming delivers partial results progressively — users see itinerary before the pipeline completes
LLM hallucination (costs, routes)	Edge Agent applies deterministic Python heuristics to detect and correct known failure patterns
Flight data is LLM-enriched, not live inventory	Deep-links to Skyscanner and Google Flights allow users to verify and book real seats
No user authentication	Each session is stateless; no trip history is persisted — acceptable for hackathon scope
Ilmu AI API dependency	Single LLM provider; no automatic failover to a secondary provider if API is down
Payment is simulated	No real financial transactions occur — card number is validated client-side but never processed
Currency API relies on CDN	Offline fallback with static rates for 17 currencies ensures the app remains functional

15. Conclusion

DaddiesTrip demonstrates that a carefully orchestrated multi-agent LLM pipeline — when combined with deterministic validation, real-time SSE streaming, and a responsive frontend — can deliver a genuinely useful, end-to-end travel planning experience from a single natural language prompt.

The system's four-agent design (Analyzer → Planner → Booking → Edge) is its core architectural insight: by giving each agent a single, narrow responsibility, the team minimised hallucination risk and maximised the granularity of progressive feedback to the user.

Built for UM Hackathon 2026 by the SugarDaddies team from Universiti Teknologi Malaysia, DaddiesTrip is a production-deployed, fully functional platform accessible at https://daddies-trip.vercel.app/ — live for evaluation at any time.

> *"SugarDaddies: We take care of everything. Especially you." — Let daddy handle the itinerary.*

SugarDaddies Team — UM Hackathon 2026
Universiti Teknologi Malaysia
Document Date: April 25, 2026